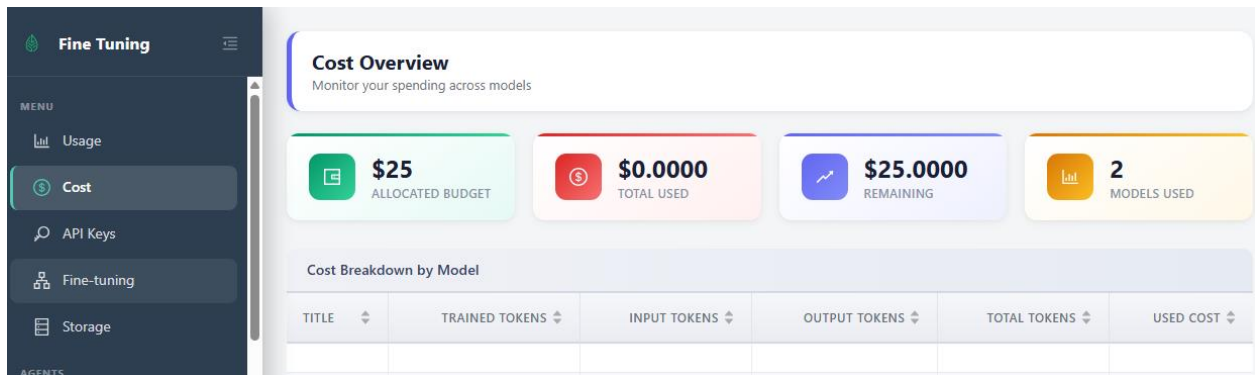


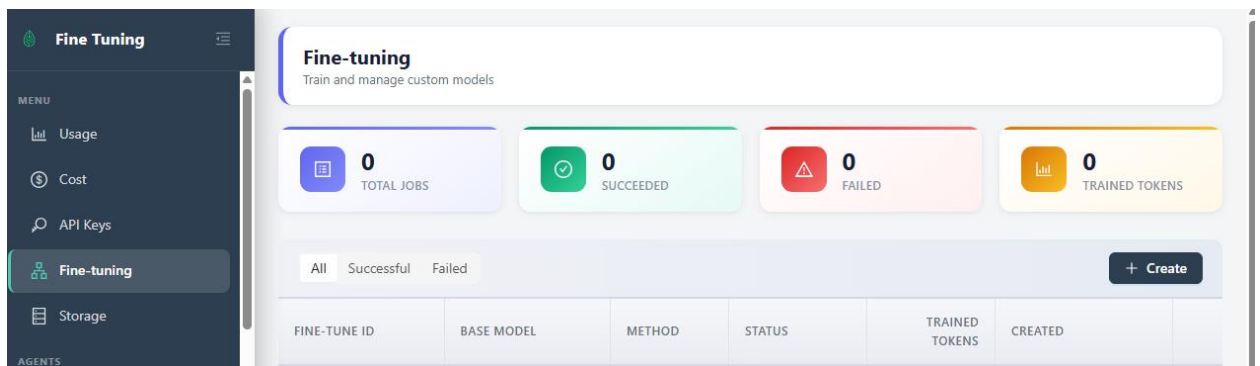
Week4 Day 2 Assignment:

Fine Tune the model and use it in web-extension and verify the result generated based on the training.

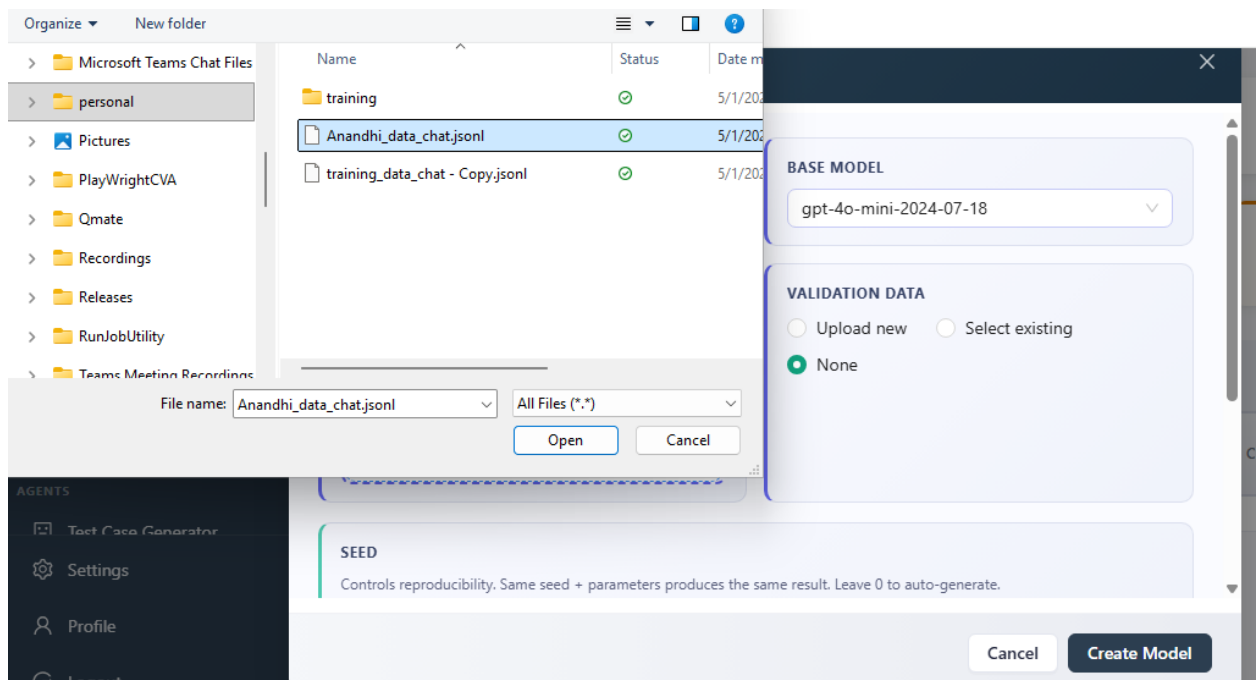
1. Login to api.testleaf.com



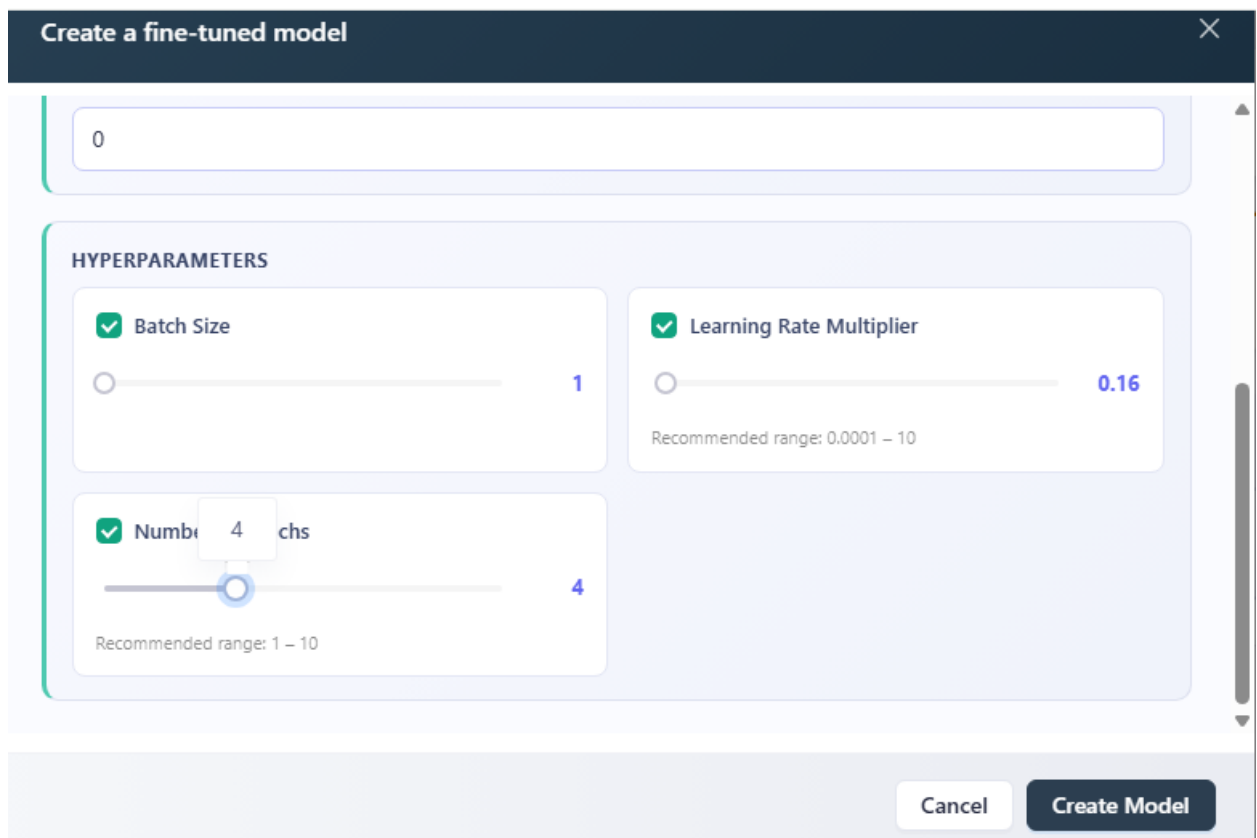
2. Select fine tuning and select the create button



3. In the popup that appears select the Method as Supervised and Model as gpt-4o-mini-2024-07-18. Upload the jsonl file we have created in day 1



4. Set the batch size as 1. Learning rate multiplier should be low which is in range 0.1 to 0.2. Number of epochs set them as 4. Then click create model



Fine-tuning

Train and manage custom models

**1**

TOTAL JOBS

**0**

SUCCEEDED

**0**

FAILED

**0**

TRAINED TOKE

AllSuccessfulFailed

+ Cre

FINE-TUNE ID	BASE MODEL	METHOD	STATUS	TRAINED TOKENS	CREATED
gpt-4o-mini-2024-07-18	gpt-4o-mini-2024-07...	supervised	Validating_files	-	01 May 2026 21:11

 Fine-tuning job created successf

5. Job is succeeded now

1

TOTAL JOBS

1

SUCCEEDED

0

FAILED

21,316

TRAINED TOKENS

All Successful Failed

+ Create

FINE-TUNE ID	BASE MODEL	METHOD	STATUS	TRAINED TOKENS	CREATED	
ft:gpt-4o-mini-2024-...	gpt-4o-mini-2024-0...	supervised	Succeeded	21,316	01 May 2026 21:11	

ft:gpt-4o-mini-2024-07-18:testleaf-2::DakPzR5J

Status

Succeeded

Job ID

ftjob-W9UoCzJbbW5etyQ47tHOO6Se

Training Method

supervised

Base Model

gpt-4o-mini-2024-07-18

Output Model

ft:gpt-4o-mini-2024-07-18:testleaf-2::DakPzR5J

Created At

01 May 2026 21:11:30

Trained Tokens

21,316

Epochs

1

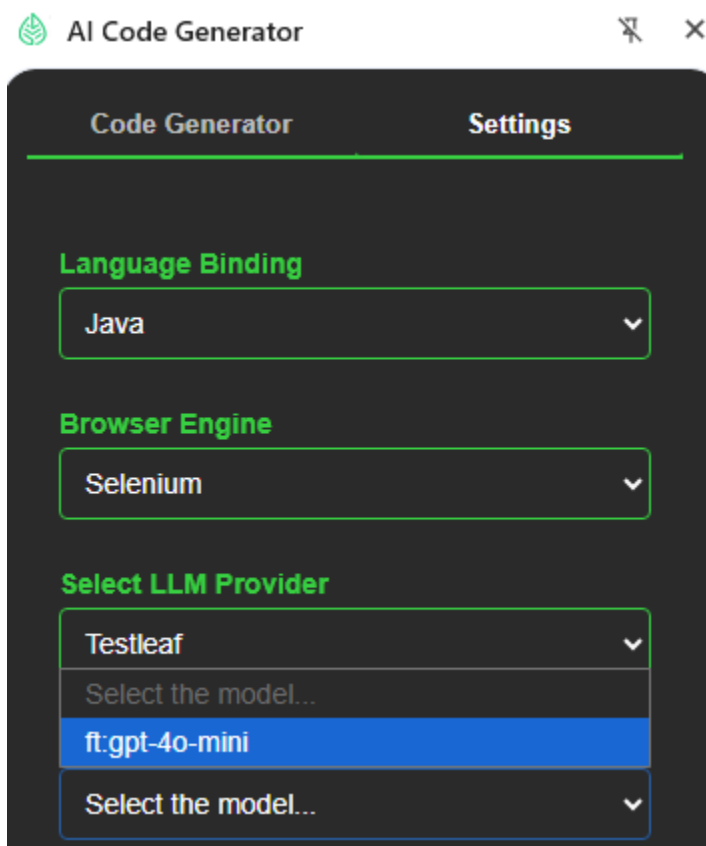
Re-run as Job

Close

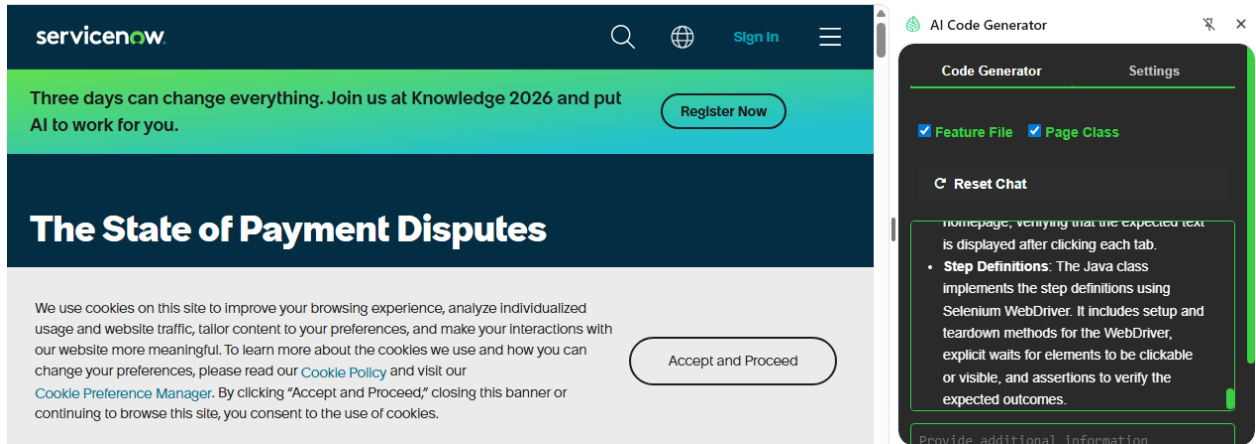
- The fine tuned model name is *ft:gpt-4o-mini-2024-07-18:testleaf-2::DakPzR5J*
- Lets feed this model option into the AI Extension we have created previously In the popup.js file

```
JS popup.js •
src > scripts > JS popup.js > document.addEventListener("DOMContentLoaded") callback > modelsByProvider
5 document.addEventListener('DOMContentLoaded', async () => {
107 // Model options by provider
108 const modelsByProvider = {
109   groq: [
110     { value: 'meta-llama/llama-4-maverick-17b-128e-instruct', label: 'meta-llama/llama-4-maverick-17b-128e-instruct' },
111     { value: 'openai/gpt-oss-120b', label: 'openai/gpt-oss-120b' },
112   ],
113   openai: [
114     { value: 'gpt-4o', label: 'GPT-4o' },
115     { value: 'gpt-4o-mini', label: 'GPT-4o Mini' },
116   ],
117   testleaf: [
118     { value: 'ft:gpt-4o-mini-2024-07-18:testleaf-2::DakPzR5J', label: 'ft:gpt-4o-mini' }
119   ],
120 ];
121
122
```

8. Readd the extension again to chrome browser and you will now see the fine tuned model and add the testleaf Api key



9. Once completed, launch any website of your choice and try to create feature file as well as pageclass



Feature: Navigation to ServiceNow Products Page

Scenario Outline: Navigate to ServiceNow Products Page

Given I open the ServiceNow homepage

When I click on the "<tab>" tab

Then I should see the "<expectedText>" displayed

Examples:

<i> tab</i>	<i> expectedText</i>	<i> </i>
<i> MyNow</i>	<i> MyNow</i>	<i> </i>
<i> Products</i>	<i> Products</i>	<i> </i>
<i> Industries</i>	<i> Industries</i>	<i> </i>
<i> Learning</i>	<i> Learning</i>	<i> </i>
<i> Support</i>	<i> Support</i>	<i> </i>
<i> Partners</i>	<i> Partners</i>	<i> </i>
<i> Company</i>	<i> Company</i>	<i> </i>
<i> Knowledge</i>	<i> Knowledge</i>	<i> </i>

```
package com.servicenow.stepdefs;
```

```
import io.cucumber.java.en.*;
```

```
import org.openqa.selenium.*;
```

```
import org.openqa.selenium.chrome.ChromeDriver;
```

```
import org.openqa.selenium.support.ui.*;
```

```

import static org.junit.Assert.assertTrue;

public class NavigationStepDefinitions {
    private WebDriver driver;
    private WebDriverWait wait;

    @io.cucumber.java.Before
    public void setUp() {
        System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");
        driver = new ChromeDriver();
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        driver.manage().window().maximize();
    }

    @io.cucumber.java.After
    public void tearDown() {
        if (driver != null) driver.quit();
    }

    @Given("I open the ServiceNow homepage")
    public void openServiceNowHomepage() {
        driver.get("https://www.servicenow.com/");
    }

    @When("I click on the {string} tab")
    public void clickOnTab(String tab) {
        WebElement tabElement =
wait.until(ExpectedConditions.elementToBeClickable(By.id("cmp-nav__tab--" + tab +
"Nav")));
        tabElement.click();
    }

    @Then("I should see the {string} displayed")
    public void verifyTabDisplayed(String expectedText) {
        WebElement displayedText =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.xpath("//span[contains(text
()),'" + expectedText + "']")));
        assertTrue(displayedText.isDisplayed());
    }

```

```
}  
}
```

Explanation:

- **Feature File:** *The Gherkin feature file defines a scenario outline for navigating to different tabs on the ServiceNow homepage, verifying that the expected text is displayed after clicking each tab.*
- **Step Definitions:** *The Java class implements the step definitions using Selenium WebDriver. It includes setup and teardown methods for the WebDriver, explicit waits for elements to be clickable or visible, and assertions to verify the expected outcomes.*